



AuditWare

X

0xbow

Smart Contract Security Review

Performed by **Auditware** for **0xbow**



auditware.io



[audit_wizard](#)



Review Summary

The Auditware team has performed a security review on Oxbow's Privacy Pools protocol. We have restricted the scope to the portions that provide mnemonic key derivation.

Privacy Pools has implemented a deterministic seed phrase generation system that derives BIP39 mnemonic phrases from Ethereum wallet signatures. This system eliminates the need for users to manually create and store traditional seed phrases while maintaining cryptographic security through EIP-712 structured signing and HKDF key derivation.

The scope consisted of changes related to the transition to the abovementioned change:

- [dev/src/utils/walletSeed.ts](#)
- [dev/src/containers/SeedPhraseForm.tsx#L171-L200](#)
- [dev/src/_tests_/walletSeed.test.ts](#)
- [pull/125/commits/7fb4589a2c1c3e996a470d3e57f683b87b784573](#)

Audit commit hash: [7fb4589](#)

Analysis of Entropy Calculations:

We performed a mathematical analysis of the entropy input into both the existing and new derivation functions. Both functions maintain a level of 128 bits of non-public entropy, requiring an average of 2^{128} attempts to brute force a mnemonic seed.

Original method - mnemonicFromEntropy(entropy):

```
mnemonic = BIP39(entropy || SHA256(entropy)[0:4_bits])

H(mnemonic) = H(entropy | algorithm)
              = H(entropy) + H(algorithm | entropy)
              = H(entropy) + 0      [algorithm is public/known]
              = H(entropy)
              = 128 bits
```

New method - deriveMnemonicFromWalletSignature(signature, address):

```
r = signature[0:32]
mnemonic = BIP39(HKDF(r, salt=address, info='privacy-pools/wallet-seed:v1', len=16))

H(mnemonic) = H(r | address, info, algorithm)
```



```
= H(r) + H(address | r) + H(info | r, address) + H(algorithm | r, address,  
info)  
  
= H(r) + 0 + 0 + 0    [address, info, algorithm are public/known]  
= H(r)  
= H(RFC6979(private_key, message_hash))  
= H(RFC6979(private_key, hash(keccak256(address))))  
= H(private_key)      [message is deterministic from public address]  
= min(256, HKDF_output_bits)  
= min(256, 128)  
= 128 bits
```



Findings Overview

Finding	Type	Risk Level	Status
<u>AW-H-01: Authentication Bypass via Insecure EIP-712 Implementation in Seed Derivation</u>	EIP-712	High	Acknowledged
<u>AW-L-01: Insufficient Client-Side Hardening</u>	Frontend	Low	Umitigated
<u>AW-L-02: Insufficient Security Test Coverage for Entropy Weakness</u>	Testing	Low	Umitigated



AW-H-01: Authentication Bypass via Insecure EIP-712 Implementation in Seed Derivation

Severity: *High*

Status: *Acknowledged*

Code:

- [dev/src/utis/walletSeed.ts#L97-L115](#)

Note:

Following discussions with the protocol team regarding the identified security concerns and user experience considerations, the team has decided to accept the acknowledged risks as part of their operational model. Users will be required to assume these risks, with the protocol committing to implement comprehensive warning systems and educational materials to ensure full user awareness. The team noted that a safer passphrase-based alternative is already available to provide users with more secure interaction options.

Description:

The Privacy Pools seed derivation system uses EIP-712 signatures as the primary authentication mechanism for generating BIP39 mnemonic phrases.

The current implementation creates EIP-712 typed data with a minimal domain containing only **name** and **version**, lacking **chainId** and **verifyingContract**.

```
// Build the EIP-712 typed data for seed derivation, committing to keccak256(address).
export function buildSeedDerivationTypedData(address: string) {
  const addrBytes = toBytes(address as `0x${string}`);
  const addressHash = keccak256(addrBytes);
  const domain = { name: 'Privacy Pools', version: '1' } as const;
  const types = {
    DeriveSeed: [
      { name: 'action', type: 'string' },
      { name: 'context', type: 'string' },
      { name: 'addressHash', type: 'bytes32' },
    ],
  } as const;
  const message = {
    action: 'Derive Account Seed',
    context: 'privacy-pools/wallet-seed:v1',
  }
```



```
addressHash: addressHash as `0x${string}`,  
} as const;  
return { domain, types, message, primaryType: 'DeriveSeed' as const };  
}
```

Signatures are verified entirely in the frontend using JavaScript cryptography, with no server-side validation or replay protection.

The system is vulnerable to domain impersonation attacks where attackers can create fake applications with identical EIP-712 domains (name, version, chainId and verifyingContract), collect user signatures, and derive the same seed phrases.

The HKDF salt (user address) provides user separation but no security against this kind of signature theft. Without expiration or server-side validation, stolen signatures remain valid indefinitely.

Successful exploitation may result in complete compromise of user privacy accounts, fund theft, and permanent transaction linkability.

Although users are expected to remain vigilant and not fall for phishing sites requesting for their wallet, it's on the protocol side to ensure their users authentication is secured and not replayable in those cases.

This issue's severity is **High**, and not Critical due to the fact that a user must first fall for a phishing site before becoming vulnerable. However, it must still be thought of thoroughly.

Recommendations:

- During the audit we have not managed to find a way to secure EIP-712 against mentioned replay attacks, and on the [EIP-712 page itself](#) no explicit protection is natively suggested by the protocol, even though the risks are mentioned. Instead, we recommend using a solution like [EIP-4361 \(SIWE\)](#) that provides built-in protection against domain impersonation, replay attacks, and requires server-side validation.
- If the protocol can't transition to an EIP-4361 or similar approach, due to business requirements towards deterministic mnemonic generation, they must be aware of the risks associated with this decision and attempt to implement as much as possible from the following:



- Origin domain check (e.g. if the attacker will create a fake domain, the signature containing it won't match expected one)
- Clear expiration & timing controls
- If possible - nonce-based replay protection - nonce must be freshly generated for each sign-in, server tracks used nonces and rejects duplicates, attacker can't reuse a stolen signature with the same nonce
- Even within EIP-712 (which is not recommended at this case) there are fields to slightly make an attacker harder (although still possible) like **chainId**, **verifyingContract** and **salt** that can be added to the domain fields.



AW-L-01: Insufficient Client-Side Hardening

Severity: **Low**

Status: **Unmitigated**

Code:

- [dev/src/containers/SeedPhraseForm.tsx#L171-L200](#)
- [dev/next.config.mjs#L32-L53](#)

Description:

During the audit of the **SeedPhraseForm.tsx** file we've come upon several missing client-side hardening features, to protect against threat vectors aiming to steal user's derived signature or browsing session. The following deficiencies were identified:

- Authentication data is saved to localStorage without an expiration
- CSP is implemented, but still allows execution of arbitrary scripts
- Not all other recommended HTTP headers are configured
- Authentication data is logged to the console

In case of a compromised CDN (e.g. supply chain attack), user downloading malicious browser extension, or similar, this allows potential threat actors to leak signature generation context or act on behalf of the user.

Recommendations:

- Add localStorage expiration for authentication data:

```
// Add utility function for expiring localStorage
const setStorageWithExpiration = (key: string, value: string, hours: number) =>
{
  const item = {
    value,
    expiration: Date.now() + (hours * 60 * 60 * 1000)
  };
  localStorage.setItem(key, JSON.stringify(item));
};

// Use in wallet authentication flow with 2-hour expiration
setStorageWithExpiration(`wallet-auth-${address}`, authData, 2);
```



- Strengthen CSP to prevent arbitrary script execution (suggestions only, fit your usecase):

```
// next.config.mjs - Enhance existing CSP
'Content-Security-Policy': [
  "frame-ancestors 'self' https://app.safe.global https://*.safe.global",
  "script-src 'self' 'nonce-[RANDOM]' https://cdn.jsdelivr.net", // Whitelist
specific CDNs only
  "object-src 'none'",
  "base-uri 'self'"
].join('; ')
```

- Add missing recommended HTTP security headers (suggestions only, fit your usecase):

```
// next.config.mjs - Add to existing headers array (lines 32-53)
{
  key: 'Cross-Origin-Opener-Policy',
  value: 'same-origin' // Block XSSLeaks & popup hijacks on wallet state
},
{
  key: 'Cross-Origin-Embedder-Policy',
  value: 'require-corp' // Block XSSLeaks & popup hijacks on wallet state
},
{
  key: 'Cross-Origin-Resource-Policy',
  value: 'cross-origin' // Block XSSLeaks & popup hijacks on wallet state
},
{
  key: 'Referrer-Policy',
  value: 'strict-origin-when-cross-origin' // Prevents leaking wallet addresses
in requests
},
{
  key: 'Permissions-Policy',
  value: 'camera=(), microphone=(), geolocation=()' // Disables risky browser
APIs used in phishing overlays
},
{
  key: 'X-Content-Type-Options',
  value: 'nosniff' // Blocks MIME-type sniffing payloads
},
{
  key: 'Cache-Control',
  value: 'no-store, no-cache, must-revalidate' // Stops wallet/session data
leaking via caches
},
{
  key: 'Strict-Transport-Security',
  value: 'max-age=31536000; includeSubDomains' // Enforces HTTPS, stops MITM
script injection
}
```



- Sanitize console logging of authentication data:

```
// SeedPhraseForm.tsx lines 195-197 - Replace sensitive logging
catch (err) {
  console.error('Wallet signature generation failed'); // Generic message only
  captureException(new Error('Signature generation error'), {
    tags: { stage: 'generate_mnemonic_wallet' },
    extra: { errorType: err.constructor.name } // Log error type, not content
  });
}
```

- Add React state cleanup on component unmount:

```
// Clear sensitive state when component unmounts
useEffect(() => {
  return () => {
    setSplitSeedPhrase(Array(12).fill(''));
    setIsHidden(true);
  };
}, []);
```



AW-L-02: Insufficient Security Test Coverage for Entropy Weakness

Severity: Low

Status: Unmitigated

Code:

- [src/ tests /walletSeed.test.ts#L13](#)

Description:

The current test suite for wallet seed generation only validates deterministic behavior using a single fixed private key (**0xaaaa...**).

The tests verify that the same mnemonic is derived consistently across 50 iterations but do not include any security-focused or randomized test cases that would detect entropy weakness vulnerabilities.

Recommended missing security test scenarios include:

- Random address pattern testing
- Entropy distribution analysis across different address types
- Statistical validation of derived seed randomness
- Collision resistance testing
- Cross-user seed correlation analysis.

The lack of these tests means that subtle security vulnerabilities like entropy bias or correlation attacks could go undetected during future development and code review processes.

While insufficient test coverage doesn't directly create exploitable vulnerabilities, it is important to enlarge the security test coverage in a privacy oriented protocol.



Recommendations:

- Implement entropy quality validation tests that measure the randomness and distribution properties of derived seeds.
- Add comprehensive security test cases covering random address attack scenarios, for example:

```
describe('random address security tests', () => {  
  it('should maintain entropy quality with random  
addresses', async () => {  
    // Test with random address patterns  
  });  
  it('should not leak entropy correlation across related  
addresses', async () => {  
    // Test for cross-user correlation vulnerabilities  
  });  
});
```

- Add tests for extreme address patterns (all zeros, all F's, repeated patterns) to validate security boundaries.





This report was produced by Auditware for 0xbow based solely on the information provided by 0xbow. Auditware provides no guarantees as to the accuracy of the contents of this report and does not make any guarantees that following the advice within will prevent security incidents or issues.

Auditware is available for questions or comments about any of the contents of this report. We can be reached at <https://auditware.io/> or by email at joe@auditware.io.



auditware.io



audit_wizard

